

Arquitectura de Chronova

Este documento explica qué es la arquitectura hexagonal, por qué se eligió para Chronova y dónde está cada cosa. Está escrito para que se entienda sin experiencia previa en el patrón.

1. La idea en una frase

Las reglas del negocio van en el centro y no dependen de nada. La tecnología (base de datos, servidor web, cifrado) va en el borde y se puede cambiar sin tocar el centro.

Esta arquitectura también se conoce como **puertos y adaptadores**, y es el nombre que mejor la describe.

2. La analogía del enchufe

Piensa en un computador portátil.

El portátil **no sabe** de dónde viene la electricidad. Puede venir de la red eléctrica, de una planta a gasolina o de un panel solar. Al portátil le da igual, porque no habla con la fuente de energía: habla con un **enchufe**, que es un contrato de forma y voltaje. Cualquier cosa que cumpla ese contrato sirve.

- El **portátil** es el dominio: la parte valiosa, la que hace el trabajo.
- El **enchufe** es el puerto: un contrato, no una implementación.
- La **red eléctrica o el panel solar** son los adaptadores: intercambiables.

En Chronova, el "portátil" son las reglas del tratamiento farmacológico: cuándo toca una toma, cómo se calcula la adherencia, quién puede ver los datos de quién. El "enchufe" son interfaces de TypeScript. El "panel solar" es PostgreSQL, que mañana podría ser MySQL sin que las reglas se enteren.

3. Las tres capas



La regla de oro: las flechas de dependencia apuntan siempre hacia adentro. El dominio no importa nada de las otras capas. Nunca.

Puedes comprobarlo tú mismo. Abre cualquier archivo de `backend/src/domain/` y mira sus importaciones: solo hay otros archivos del propio dominio. No aparece `express`, ni `pg`, ni `bcrypt`, ni una sola librería externa.

4. Qué va en cada capa

Dominio — `backend/src/domain/`

Las reglas que serían ciertas aunque Chronova fuera una libreta de papel.

Archivo	Regla que protege
<code>shared/FechaLocal.ts</code>	Un día del calendario no es un instante: no se desplaza entre husos
<code>shared/ZonaHoraria.ts</code>	"Las 8 de la mañana" son las 8 donde vive el paciente
<code>medicamento/Frecuencia.ts</code>	¿Toca este medicamento hoy?
<code>medicamento/Stock.ts</code>	El inventario nunca baja de cero; avisa al llegar al umbral
<code>medicamento/Medicamento.ts</code>	No puede haber horarios repetidos; un medicamento suspendido no genera agenda
<code>toma/Toma.ts</code>	Una toma no se puede registrar dos veces; máximo 3 aplazamientos
<code>toma/ResumenDeAdherencia.ts</code>	Adherencia = tomadas ÷ resueltas. Buena a partir del 80%
<code>vinculo/Vinculo.ts</code>	Un cuidador solo accede si el paciente aceptó
<code>recuperacion/SolicitudDeRecuperacion.ts</code>	El código caduca, sirve una vez y admite cinco intentos

Cada entidad se construye con una fábrica que la valida al nacer — `Medicamento.crear()`, `Paciente.registrar()`, `Toma.programar()`, `Vinculo.solicitar()` — y todas tienen `desdePlano()` (se reconstruye desde la base de datos) y `aPlano()` (se convierte en datos simples para guardar o enviar). Los constructores son privados: no hay forma de fabricar una entidad en un estado inválido.

Aplicación — `backend/src/application/`

Los **casos de uso**: cada acción que el sistema sabe hacer, una clase con un método `ejecutar()`.

Un caso de uso orquesta pero no decide. Por ejemplo, `RegistrarToma`:

1. Busca la toma.
2. Pide permiso a la política de acceso.
3. Le dice a la entidad `Toma` que se confirme (la entidad valida si puede).
4. Le dice al `Medicamento` que descuente inventario (la regla del descuento está en la entidad).
5. Guarda y avisa si el stock quedó bajo.

Ninguno de esos pasos contiene una regla de negocio: todas están dentro de las entidades.

Aquí también viven los **puertos**, en `application/ports/`:

Puerto	Contrato	Por qué existe
Reloj	"dame la hora"	Para poder congelar el tiempo en las pruebas
GeneradorDeIds	"dame un id nuevo"	Para tener ids predecibles en las pruebas
CifradorDeContrasenas	"cifra / verifica"	Para cambiar bcrypt por otro sin tocar nada más
ServicioDeTokens	"emite / verifica sesión"	Para que el dominio no sepa qué es un JWT
Notificador	"avisa esto a esta persona"	Consola, notificaciones push, o ambas: se elige en el contenedor
EnviadorDeCorreo	"manda este correo"	Consola durante el desarrollo, un proveedor real en producción
GeneradorDeCodigos	"dame un número impredecible"	Para fijarlo en las pruebas, y para no usar <code>Math.random</code> en producción

Los puertos de persistencia (`RepositorioDeMedicamentos` , etc.) están en el dominio, porque es el dominio quien define qué necesita.

Infraestructura — `backend/src/infrastructure/`

Todo lo que se puede reemplazar.

Carpeta	Qué contiene
<code>persistence/postgres/</code>	Todas las consultas SQL del proyecto. Absolutamente todas.
<code>persistence/in-memory/</code>	Los mismos repositorios, con arreglos en memoria
<code>security/</code>	bcrypt y JWT
<code>system/</code>	Reloj del sistema y generador de UUID
<code>notificaciones/</code>	Los tres notificadores (consola, push de Expo y el compuesto) y el cliente HTTP de Expo
<code>correo/</code>	Los dos adaptadores de correo: consola y proveedor real
<code>http/</code>	Express: rutas, middlewares, validación, traducción de errores

5. Por qué esto importa: seis pruebas concretas

Prueba 1: la aplicación corre sin base de datos

Con `PERSISTENCE=memory` en el `.env`, Chronova funciona completa sin instalar PostgreSQL. Registro, medicamentos, agenda, adherencia, cuidadores: todo.

Eso solo es posible porque el dominio nunca supo que existía una base de datos. Lo que cambia el usuario es una línea del `.env`; en `contenedor.ts` la elección ya está escrita, y es el único sitio de todo el proyecto donde aparece.

Prueba 2: las pruebas corren en milisegundos

Las 146 pruebas de `backend/tests/` ejercitan la aplicación entera —incluidos los casos de uso completos— y terminan en menos de dos segundos, sin base de datos, sin servidor y sin red.

Con la arquitectura del MVP anterior, donde la lógica vivía dentro de las rutas de Express y hablaba directamente con PostgreSQL, cada prueba habría necesitado una base de datos levantada, datos sembrados y limpieza posterior. En la práctica, eso significa que nadie escribe pruebas.

Prueba 3: el tiempo se puede manipular

Para probar "¿qué pasa si el paciente no confirma la toma en dos horas?" no hay que esperar dos horas: se inyecta un `RelojFijo` y se mueve. Mira `backend/tests/use-cases/casosDeUso.test.ts`, la prueba del cierre automático.

Prueba 4: las notificaciones push no obligaron a tocar nada de lo que ya existía

El puerto `Notificador` existía desde el primer día con un solo adaptador que escribía en consola. Añadir el envío real consistió en escribir un adaptador nuevo (`NotificadorExpoPush`) y enchufarlo en `contenedor.ts`.

Conviene ser preciso con lo que eso significa, porque es comprobable con `git show`. **Ninguna entidad ni caso de uso ya existente se modificó:** `CerrarTomasVencidas` sigue diciendo «avisa esto a esta persona» sin saber que ahora eso llega a un teléfono, y no cambió ni una línea. Lo que sí se añadió fue material nuevo —el agregado `Dispositivo`, dos casos de uso para dar de alta y de baja un teléfono, y el cableado en el contenedor—, porque guardar en qué aparatos hay que entregar el aviso es una responsabilidad que antes no existía en el sistema.

Esa es la diferencia que importa: crecer añadiendo piezas nuevas, sin volver a abrir las que ya funcionaban.

Y como los adaptadores cumplen el mismo contrato, se pueden combinar: `NOTIFICACIONES=ambos` usa un `NotificadorCompuesto` que contiene a los dos y cumple, él también, la misma interfaz.

Prueba 5: el resultado no depende del reloj del servidor

La suite completa se ejecuta bajo seis zonas horarias distintas —de Kiritimati (UTC+14) a Anchorage (UTC−9) — y da exactamente el mismo resultado:

TZ=UTC	npm test	# 146 passed
TZ=America/Bogota	npm test	# 146 passed
TZ=Asia/Tokyo	npm test	# 146 passed

Esto no era así al principio. El dominio usaba objetos `Date` para representar días del calendario, y un `Date` siempre es un instante que se lee en la zona del proceso. La consecuencia: con el servidor en UTC, la pastilla de las 08:00 de una paciente colombiana se agendaba a las 03:00 de su madrugada.

La corrección fue conceptual, no un parche: separar dos ideas que estaban mezcladas.

- **FechaLocal** es un día del calendario. No tiene hora, así que no puede desplazarse.
- **ZonaHoraria** traduce entre la *hora de pared* del paciente y el *instante* que se guarda y dispara la alarma.

Con esas dos piezas, el dominio dejó de tener opinión sobre dónde corre el servidor.

Prueba 6: una pantalla nueva sin una línea de servidor

La pantalla de detalle del paciente —donde el cuidador ve la agenda del día, el tratamiento y las tomas que se saltaron— se añadió **sin modificar un solo archivo de `backend/src/`**. Se puede comprobar en el historial de Git: el único archivo del backend que cambió en ese commit fue el de pruebas.

No fue suerte. Los casos de uso `ObtenerAgendaDelDia`, `ListarMedicamentos` y `ConsultarHistorial` ya recibían un `pacienteId`, y ya pasaban por `PoliticaDeAcceso` antes de devolver nada. Estaban escritos desde el principio pensando en que quien pregunta puede no ser el dueño de los datos.

Vale la pena señalar dónde vive esa decisión. La pantalla esconde el botón de confirmar cuando el cuidador no tiene el permiso, pero **eso es cortesía, no seguridad**: esconder un botón no impide llamar al endpoint. Lo que de verdad protege la información clínica es que el servidor se niegue, y por eso las pruebas de ese permiso están en `backend/tests/`, no en la app.

6. El punto donde todo se conecta

`backend/src/contenedor.ts` es el **composition root**: el único archivo del backend que sabe qué implementación concreta se usa para cada puerto.

```
if (entorno.persistencia === 'postgres') {
  repositorios = { medicamentos: new RepositorioDeMedicamentosPostgres(pool), ... };
} else {
  repositorios = { medicamentos: new RepositorioDeMedicamentosEnMemoria(), ... };
}
```

Migrar a MySQL sería: escribir `RepositorioDeMedicamentosMySQL` cumpliendo la misma interfaz, y agregar una rama a este `if`. Ni el dominio ni los casos de uso ni las rutas cambian una sola línea.

7. La app móvil sigue la misma idea

`mobile/src/` está organizado igual:

Carpeta	Qué es
<code>dominio/</code>	Los tipos y los puertos (<code>ApiDeChronova</code> , <code>AlmacenDeSesion</code> , <code>ProgramadorDeAlarmas</code>)
<code>infraestructura/</code>	El cliente HTTP, AsyncStorage, las alarmas de Expo
<code>ui/</code>	El sistema de diseño y los componentes
<code>ui/contexto/SesionContexto.tsx</code>	El composition root de la app

Las pantallas de `mobile/app/` nunca llaman a `fetch` directamente. Piden `api` al contexto y usan el puerto. Si mañana la comunicación fuera por WebSocket o GraphQL, solo cambiaría `ClienteChronova.ts`.

8. Dónde poner cada cosa nueva

Cuando agregues una funcionalidad, esta tabla te dice dónde va:

Lo que quieres agregar	Dónde va
Una regla nueva ("no más de 8 medicamentos activos")	Entidad del dominio
Un concepto nuevo con reglas propias (una cita médica)	Nueva carpeta en <code>domain/</code>
Una acción nueva del sistema ("exportar informe")	Nuevo caso de uso en <code>application/use-cases/</code>
Una consulta SQL	Solo en <code>infraestructura/persistence/postgres/</code>
Un endpoint nuevo	<code>infraestructura/http/routes/</code> + su esquema Zod
Una pantalla nueva	<code>mobile/app/</code>

Señal de alarma: si te encuentras escribiendo `if` con reglas de negocio dentro de un archivo de `routes/`, esa lógica está en el lugar equivocado. Su sitio es un caso de uso o una entidad.

9. Las dos validaciones (y por qué no sobra ninguna)

Chronova valida los datos dos veces, a propósito:

1. **Zod, en el borde HTTP** (`http/dtos/esquemas.ts`): comprueba la *forma*. ¿Vino el campo? ¿Es texto? ¿El número es entero? Da mensajes rápidos y precisos al usuario.

2. **El dominio:** comprueba el *significado*. ¿Esa hora existe? ¿La fecha de fin es posterior al inicio? ¿Este horario está repetido?

¿Por qué las dos? Si solo se validara en HTTP, cualquier otro punto de entrada —una tarea programada, el script de datos de ejemplo, un futuro adaptador— se saltaría las reglas. Si solo se validara en el dominio, los mensajes llegarían tarde y peor explicados.

Cada capa valida lo que le corresponde. Esa duplicación aparente es en realidad defensa en profundidad.